

Practical Session 11

Task 1

Develop a multithreaded counter application using POSIX threads (pthread_create, pthread_join). Demonstrate race conditions by allowing multiple threads to update a shared variable concurrently.

```
j_rakshitha@RAKSHITHA:~$ nano practical11_task1_race.c
j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra -pthread practical11_task1_race.c -o practical11_task1_race
j_rakshitha@RAKSHITHA:~$ ./practical11_task1_race
=====
Practical Session 11 - Task 1
Multithreaded Counter - Race Condition
=====
Expected counter value: 4000000
Actual counter value : 1302254
Result: Race condition observed
j_rakshitha@RAKSHITHA:~$
```

```
j_rakshitha@RAKSHITHA: ~ × + ▾
GNU nano 8.7.1 practical11_task1_race.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ITERATIONS 1000000

int counter = 0;

void *increment_counter(void *arg)
{
    (void)arg;

    for (int i = 0; i < ITERATIONS; i++)
    {
        counter++;
    }

    return NULL;
}

int main(void)
{
    pthread_t threads[NUM_THREADS];

    printf("=====\n");
    printf("Practical Session 11 - Task 1\n");
    printf("Multithreaded Counter - Race Condition\n");
    printf("=====\n");

    for (int i = 0; i < NUM_THREADS; i++)
    {
        if (pthread_create(&threads[i], NULL,
                          increment_counter, NULL) != 0)
        {
            perror("pthread_create");
            return 1;
        }
    }

    for (int i = 0; i < NUM_THREADS; i++)
    {
        if (pthread_join(threads[i], NULL) != 0)
        {
            perror("pthread_join");
        }
    }

    printf("Counter: %d\n", counter);
}

[ Read 65 lines ]
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line
```

```

        return 1;
    }
}

printf("\nExpected counter value: %d\n",
      NUM_THREADS * ITERATIONS);

printf("Actual counter value : %d\n", counter);

if (counter == NUM_THREADS * ITERATIONS)
{
    printf("Result: Correct\n");
}
else
{
    printf("Result: Race condition observed\n");
}

return 0;
}
|

```

Task 2

Modify the previous program using mutex locks (`pthread_mutex_lock`, `pthread_mutex_unlock`) and compare the outputs with the race-condition version. Analyze the impact of synchronization.

```

j_rakshitha@RAKSHITHA:~$ nano practical11_task2_mutex.c
j_rakshitha@RAKSHITHA:~$ gcc -Wall -Wextra -pthread practical11_task2_mutex.c -o practical11_task2_mutex
j_rakshitha@RAKSHITHA:~$ ./practical11_task2_mutex
=====
Practical Session 11 - Task 2
Multithreaded Counter - Mutex
=====

Expected counter value: 4000000
Actual counter value : 4000000
Result: Correct - mutex synchronization successful
j_rakshitha@RAKSHITHA:~$ ^C
j_rakshitha@RAKSHITHA:~$

```

```
j_rakshitha@RAKSHITHA: ~ × + v
GNU nano 8.7.1 practical11_task2_mutex.c
#include <stdio.h>
#include <stdlib.h>
#include <pthread.h>

#define NUM_THREADS 4
#define ITERATIONS 1000000

int counter = 0;

pthread_mutex_t counter_mutex = PTHREAD_MUTEX_INITIALIZER;

void *increment_counter(void *arg)
{
    (void)arg;

    for (int i = 0; i < ITERATIONS; i++)
    {
        pthread_mutex_lock(&counter_mutex);

        counter++;

        pthread_mutex_unlock(&counter_mutex);
    }

    return NULL;
}

int main(void)
{
    pthread_t threads[NUM_THREADS];

    printf("=====\n");
    printf("Practical Session 11 - Task 2\n");
    printf("Multithreaded Counter - Mutex\n");
    printf("=====\n");

    for (int i = 0; i < NUM_THREADS; i++)
    {
        if (pthread_create(&threads[i], NULL,
                          increment_counter, NULL) != 0)
        {
            perror("pthread_create");
            return 1;
        }
    }

    [ Read 73 lines ]
^G Help      ^O Write Out  ^F Where Is   ^K Cut        ^T Execute    ^C Location
^X Exit      ^R Read File  ^\ Replace    ^U Paste      ^J Justify    ^_ Go To Line
```

```

    }
}

for (int i = 0; i < NUM_THREADS; i++)
{
    if (pthread_join(threads[i], NULL) != 0)
    {
        perror("pthread_join");
        return 1;
    }
}

printf("\nExpected counter value: %d\n",
        NUM_THREADS * ITERATIONS);

printf("Actual counter value : %d\n", counter);

if (counter == NUM_THREADS * ITERATIONS)
{
    printf("Result: Correct - mutex synchronization successful\n");
}
else
{
    printf("Result: Incorrect\n");
}

pthread_mutex_destroy(&counter_mutex);

return 0;
}
|

```

[^]G Help
[^]X Exit

[^]O Write Out
[^]R Read File

[^]F Where Is
[^]\ Replace

[^]K Cut
[^]U Paste

[^]T Execute
[^]J Justify

[^]C Location
[^]/ Go To Line